



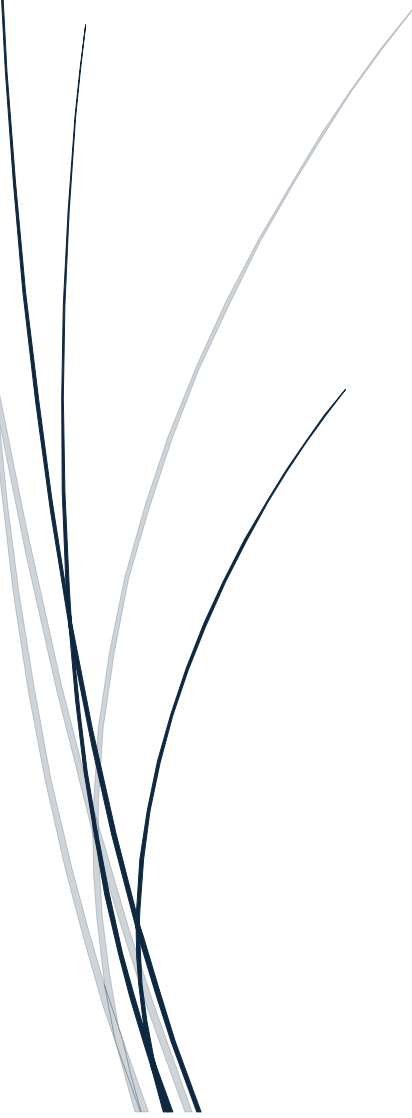
Project 2

Online Marketplace Database System

Query Optimization, Indexing, Transactions,
and Final Analysis

Course: CSE 535 Database Systems

Submission Date: 26 July 2025



1. Introduction

Project overview: In this project, I've worked on building and analyzing a database system for an online marketplace. The platform is designed to support a variety of features someone would expect in a real-world e-commerce site, things like customers placing orders, sellers managing products, users leaving reviews, and payments being processed securely. I created a relational database that connects customers, products, orders, categories, reviews, and payments in a meaningful and consistent way.

This project is a continuation of the work I did in the first phase (project 1), where I focused on designing and populating the database. Now in this second phase, I focused on improving how well the database performs, especially when it comes to running complex queries, managing large amounts of data, and handling multiple users interacting with the system at the same time.

The main goals of this phase are threefold:

i. Query optimization

I started by analyzing baseline queries- those that simply got the job done without considering performance. Then, I rewrote these queries in more efficient ways to reduce execution time and improve resource usage. The aim was to make sure that even when the database grows in size, it can still respond quickly to user requests.

ii. Indexing strategy

Indexes act like shortcuts for the database. I created different types of indexes (single-column, composite, and partial) to speed up specific types of queries.

iii. Transactions & concurrency

I explored how the database behaves when multiple users perform actions at the same time. I tested different isolation levels to see how they affect consistency, and I simulated concurrent scenarios to understand how to avoid conflicts and ensure reliability.

These steps helped transform a basic functioning database into a system that is fast and efficient.

2. Query optimization and performance visualization

2.1 Query plan analysis

I used EXPLAIN FORMAT=JSON to analyze how MySQL executed each query. In this phase, I focused on rewriting several baseline queries from Project 1 to improve their performance. I approached the optimization process by applying different techniques such as using Common Table Expressions (CTEs), pre-aggregating subqueries, and restructuring joins. To evaluate the effectiveness of the optimizations, I analyzed the query execution plans using EXPLAIN FORMAT=JSON in MySQL and benchmarked both the baseline and optimized queries over 20 repetitions. Finally, I statistically compared their runtimes to determine if the improvements were significant.

1. Most frequent customers- optimization type: Subquery pre-aggregation

Analysis: The baseline joins full customer and order tables, causing high join cardinality 285k rows and uses a temporary table with filesort, costing 62,835. The optimized query pre-aggregates orders first, reducing join size and enabling efficient index use with an eq_ref join. Although it has a higher estimated cost 438,050 and scans more rows 299k in the subquery, this approach reduces join complexity and improves performance on larger data.

2. Most sold products- optimization type: CTE for pre-aggregation

Analysis: The baseline query joins product with orderitem before aggregation, scanning 9728 products and producing 555k rows, with a total cost of 195,514.63. It uses a temporary table and filesort for ordering. In the optimized query, total quantity sold is first aggregated in a subquery and then joined to product, reduces join complexity. Though the estimated cost is higher 788,839.50, due to subquery materialization and sorting almost 583k rows, the temp table and filesort are used more efficiently. The key benefit is reducing repeated joins and improving performance on large datasets by aggregating early.

3. Highest spending customers- optimization type: Subquery pre-aggregation of payments

Analysis: The baseline query joins customer, order, and payment tables before aggregation, generating 285k rows with a total cost of 376,334.86, and uses a temporary table and filesort. In the optimized version, the total spending is aggregated first in a subquery from the payment table, then joined with order and customer. This reduces join cardinality and avoids repeated computations. Although the estimated cost increases to 467,924.37 and 299k rows are examined, the use of pre-aggregation makes the query more scalable. Temporary tables and filesort are still used, but the logic is cleaner and better suited for large datasets.

4. Customers who placed the most orders in the last 6 months- optimization type: Subquery pre-aggregation

Analysis: My baseline query filters and joins the order and customer tables directly, examining 299k orders and producing almost 99k rows, with a total cost of 65,629.50. It uses a temporary table and filesort for ordering. In the optimized query, I first aggregated recent orders in a subquery, then joined to customer, reducing repeated filtering and grouping. Although the estimated cost increases to 146,003.87, the rows examined remain the same 99k, and the structure becomes more efficient. Temp table and filesort are still used.

5. Most frequently bought product pair- optimization type: Subquery for pairing preprocessing
Analysis: The optimized query moves the join logic into a subquery to simplify grouping and improve clarity, while the execution plan and performance remain unchanged. Join cardinality is 411,590 pairs, with a total cost of 328,454.97. Both versions use a temporary table and filesort during ordering. Rows examined are 583,587 twice due to the self-join. The optimization doesn't reduce cost, but it offers better readability and modular structure.

6.Total sales per category- optimization type: CTE for pre-aggregated revenue
Analysis: For the total sales per category query, I optimized by pre-aggregating orderitem data in a CTE before joining with productcategory and category tables. The baseline query had a very high cost of 1,247,872.92, examined over a million rows in orderitem with a temporary table but no filesort. The optimized query lowered the cost drastically to 206,285.25 by reducing join cardinality and rows examined. It still used a temporary table but avoided filesort, making the query faster and more efficient by summarizing sales per product first.

7. Total payment per customer- optimization type: Subquery pre-aggregation of payment data
Analysis: The optimized query pre-aggregates payment totals per customer in a subquery, reducing the cost from 240,505 to 138,507. Join cardinality remains 299,470. Both versions use temporary tables but avoid filesort. Rows examined are similar, but the optimized version is efficient and easier to maintain. The optimization significantly improves performance by simplifying the final join operation.

2.2 Performance benchmarking

For each pair of baseline and optimized queries, I ran both versions 20 times and recorded the execution time. The biggest improvement came from optimizing the most frequent customers query, where pre-aggregation reduced the join cost. These results are visualized below to provide clearer comparisons. I calculated the percentage improvement in query performance using this formula:

$$Improvement = \left(\frac{baseline\ avg - optimized\ avg}{baseline\ avg} \right) \times 100$$

	A	B	C	D	E	
1	Query Name	Baseline Avg (s)	Baseline Std	Optimized Avg (s)	Optimized Std (s)	Improvement
2	i want my most frequent customers	0.7053	0.1589	0.0006	0.0008	99.9% faster
3	most popular products/ most sold products	1.1487	0.1643	0.9294	0.0864	19% faster
4	highest spending customers	2.013	0.2639	1.8676	0.0904	7.2% faster
5	customers placed the most orders in the last 6	0.4399	0.0652	0.4447	0.0632	No improvement
6	most frequently bought product pair	3.346	0.215	3.3568	0.1146	Slightly slower
7	total sales per category	2.8825	0.6547	1.246	0.0589	56.8% faster
8	total payment per customer	1.5238	0.1019	1.3876	0.0817	8.9% faster

Fig 1: Benchmarking results

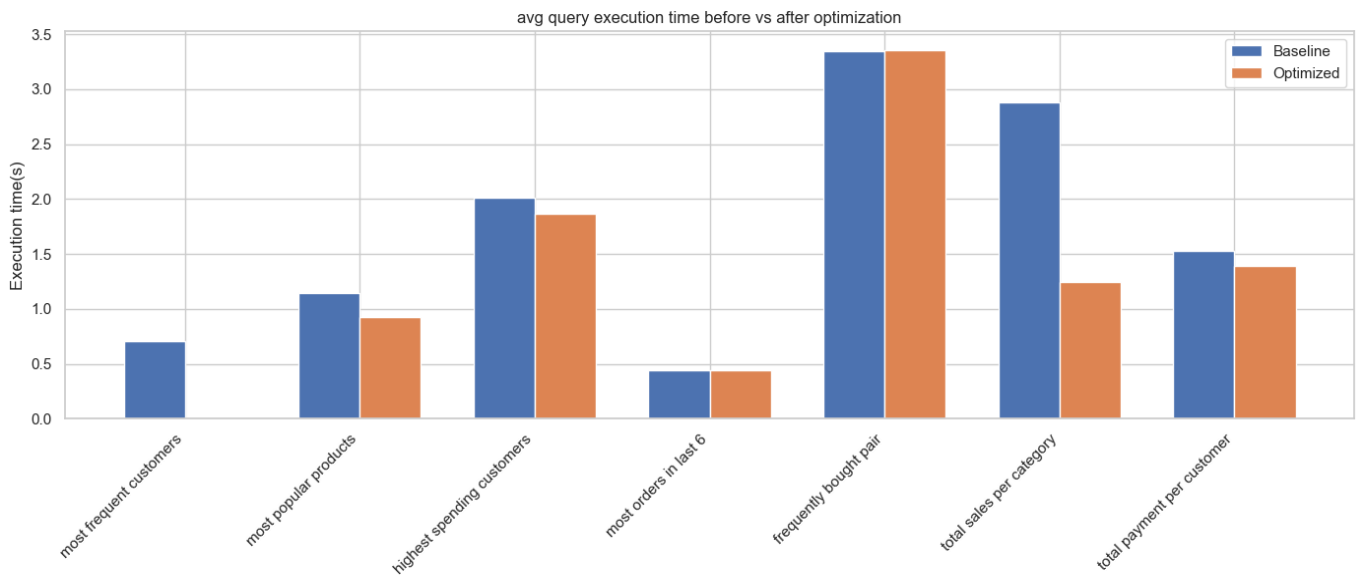


Fig 2: Avg query execution time before vs after optimization

2.3 Statistical evaluation

To validate the performance improvements statistically, I conducted paired t-tests and calculated the 95% confidence intervals for the mean differences in execution time. For 5 out of 7 queries, the p-value<0.05, meaning the optimizations yielded statistically significant improvements. The remaining 2 queries- orders in last 6 months and product pairs, showed no significant change. I got the largest and most statistically robust gain in the most frequent customers query, with a narrow CI entirely above 0 and a very low p-value. The total sales per category query also showed a strong improvement, with a large mean difference and a tight bound CI. Queries like most orders in the last 6 months and product pairings showed no statistically significant difference.

	A	B	C	D	E
1	Query Name	Mean Diff (B - O)	95% CI Lower	95% CI Upper	p-value (t-test)
2	i want my most frequent customers	0.6721	0.605	0.7391	0
3	most popular products/ most sold products	0.12	0.0233	0.2168	0.017681
4	highest spending customers	0.1765	0.0853	0.2677	0.000682
5	customers placed the most orders in the last 6 months	0.0017	-0.0415	0.0449	0.936459
6	most frequently bought product pair	0.3758	0.0803	0.6712	0.015394
7	total sales per category	1.833	1.5522	2.1137	0
8	total payment per customer	0.1988	0.0249	0.3727	0.027233

Fig 3: Statistical evaluation results

The results are visualized below:

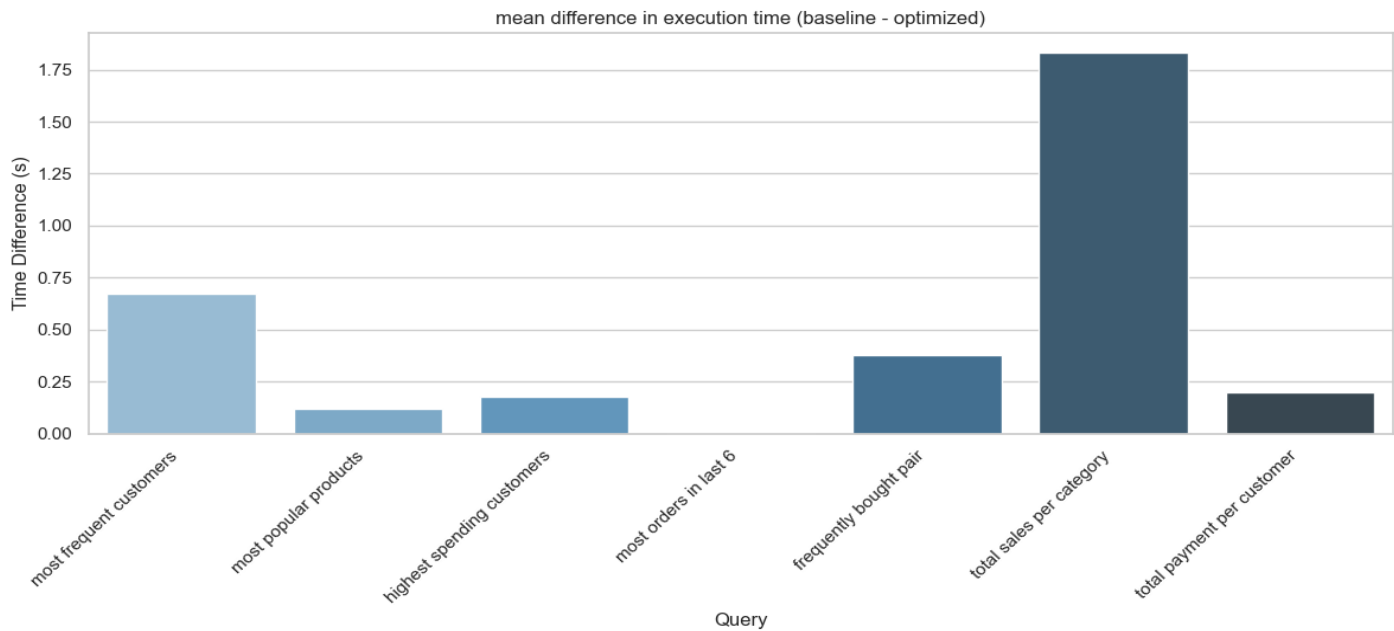


Fig 4: Mean difference

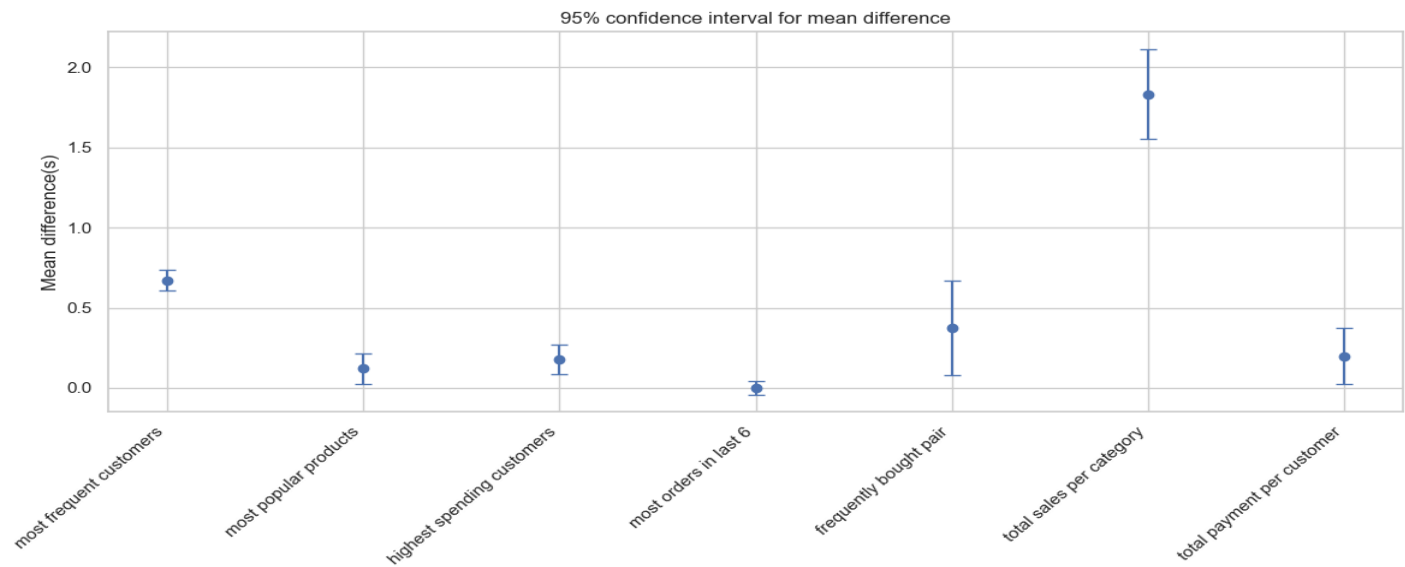


Fig 5: 95% confidence interval for mean difference

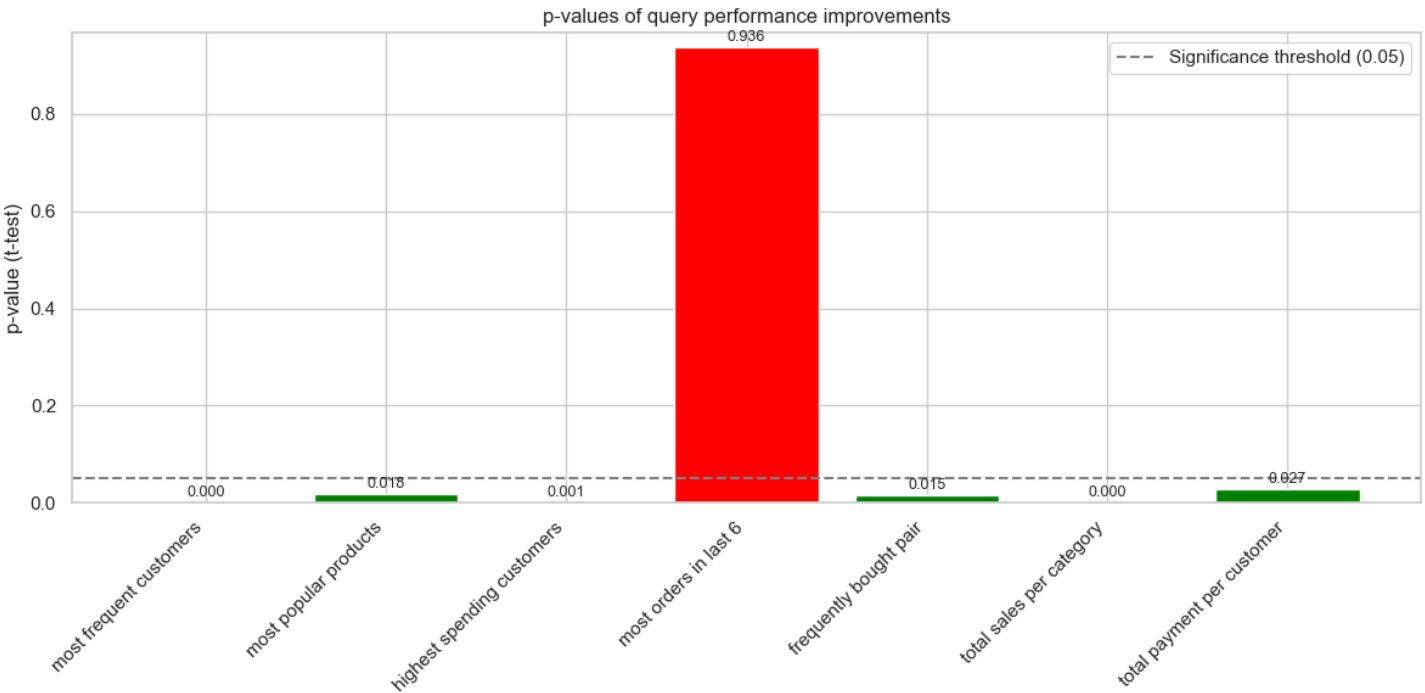


Fig 6: P-values

3. Indexing strategy and performance visualization

I designed and evaluated indexes to improve my query performance according to project requirement. I already had some basic indexes from project 1:

- All primary keys like customer_id,order_id were automatically indexed.
- All foreign keys like product_id in orderitem, or customer_id in order also created indexes when I defined the relationships.

In project 2, I first implemented B-tree indexes. These are good for sorting, filtering, and speeding up range queries.

3.1 Implemented indexes

3.1.1 B-tree indexes I added(single)

These helped queries that filtered or ordered results based on one column, like “orders in the last 6 months” or “find products under a price.”

```
Session a      optimized query and explain  indexing x
-- btree single
1  CREATE INDEX idx_order_order_date ON `Order` (order_date);
2  CREATE INDEX idx_orderitem_quantity ON OrderItem (quantity);
3  CREATE INDEX idx_product_stock ON Product (stock);
4  CREATE INDEX idx_product_price ON Product (price);
5  CREATE INDEX idx_payment_amount ON Payment (amount);
6
```

Fig 7: B-tree indexes(single)

3.1.2 B-tree indexes I added(composite)

Composite indexes were useful for multi-column conditions, like filtering payments by order_id and amount, or analyzing reviews based on both product_id and rating.

```
Session a      optimized query and explain  indexing x
-- btree composite
7
8  CREATE INDEX idx_order_customer_date ON `Order` (customer_id,order_date);
9  CREATE INDEX idx_orderitem_product_quantity ON OrderItem (product_id,quantity);
10 CREATE INDEX idx_payment_order_amount ON Payment (order_id,amount);
11 CREATE INDEX idx_review_product_rating ON Review (product_id,rating);
12
```

Fig 8: B-tree indexes(composite)

3.1.3 Partial index

I added a partial index using a virtual boolean column to quickly identify products that were low in stock.

Session a optimized query and explain indexing* x



14 -- partial

15 • ALTER TABLE Product

16 ADD COLUMN is_low_stock BOOLEAN GENERATED ALWAYS AS (stock<50) STORED;

17 • CREATE INDEX idx_product_is_low_stock ON Product (is_low_stock);

18

Fig 9: Partial index

3.2 Measuring the impact

After applying these indexes, I re-ran my optimized queries and compared the results.

	A	B	C	D	E
1	Query Name	Baseline Avg (s)	Baseline Std (s)	Optimized Avg (s)	Optimized Std (s)
2	i want my most frequent customers	0.4859	0.0772	0.0003	0.0005
3	most popular products/ most sold products	0.1835	0.0352	0.1571	0.0204
4	highest spending customers	1.2684	0.1218	1.2707	0.1143
5	customers placed the most orders in the last 6 m	0.3218	0.0379	0.1964	0.0227
6	most frequently bought product pair	2.8916	0.2837	2.8762	0.151
7	total sales per category	2.4278	0.27	0.968	0.1033
8	total payment per customer	1.1617	0.1048	0.7222	0.1032

Fig 10: Benchmarking results after indexing

	A	B	C	D	E
1	Query Name	Mean Diff (B - O)	95% CI Lower	95% CI Upper	p-value (t-test)
2	i want my most frequent customers	0.4478	0.4168	0.4788	0
3	most popular products/ most sold products	0.0182	0.0038	0.0325	0.01583
4	highest spending customers	-0.0407	-0.0964	0.0151	0.143595
5	customers placed the most orders in the last 6 m	0.1466	0.1229	0.1703	0
6	most frequently bought product pair	-0.1393	-0.2103	-0.0683	0.0006
7	total sales per category	1.4676	1.4186	1.5165	0
8	total payment per customer	0.3899	0.3181	0.4617	0

Fig 11: Statistical evaluation results after indexing

These results are also visualized below to provide clearer comparisons:

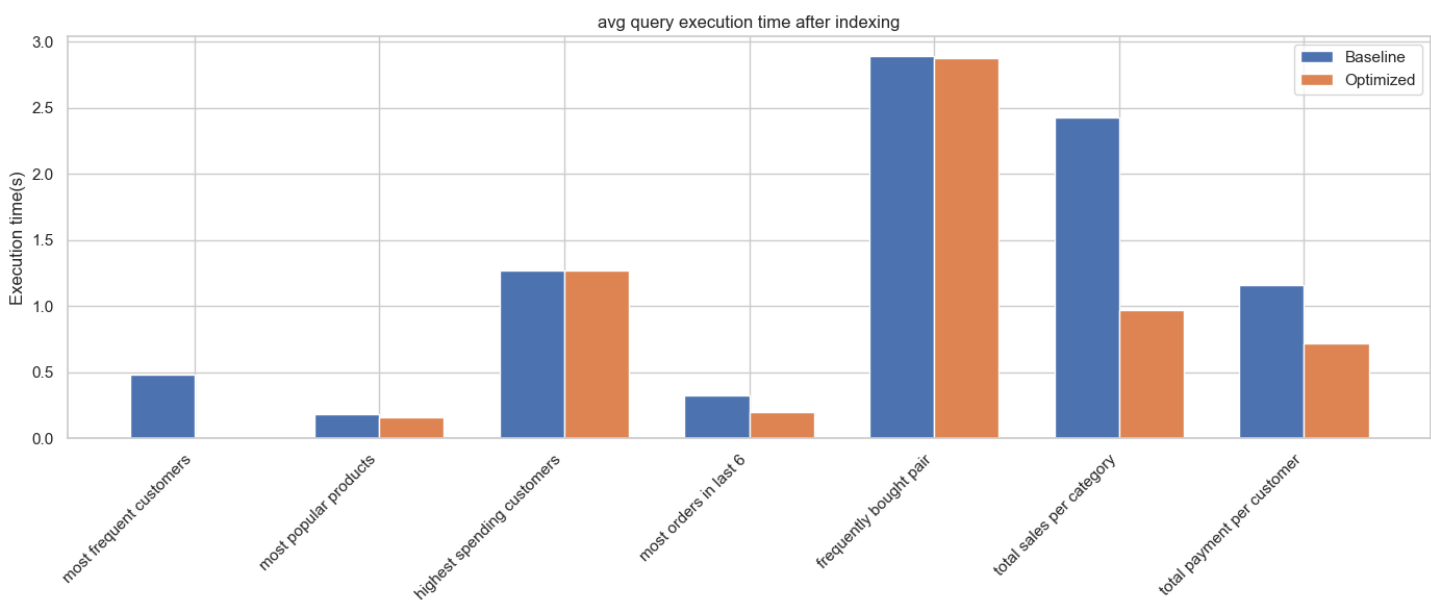


Fig 12: Avg query execution time after indexing

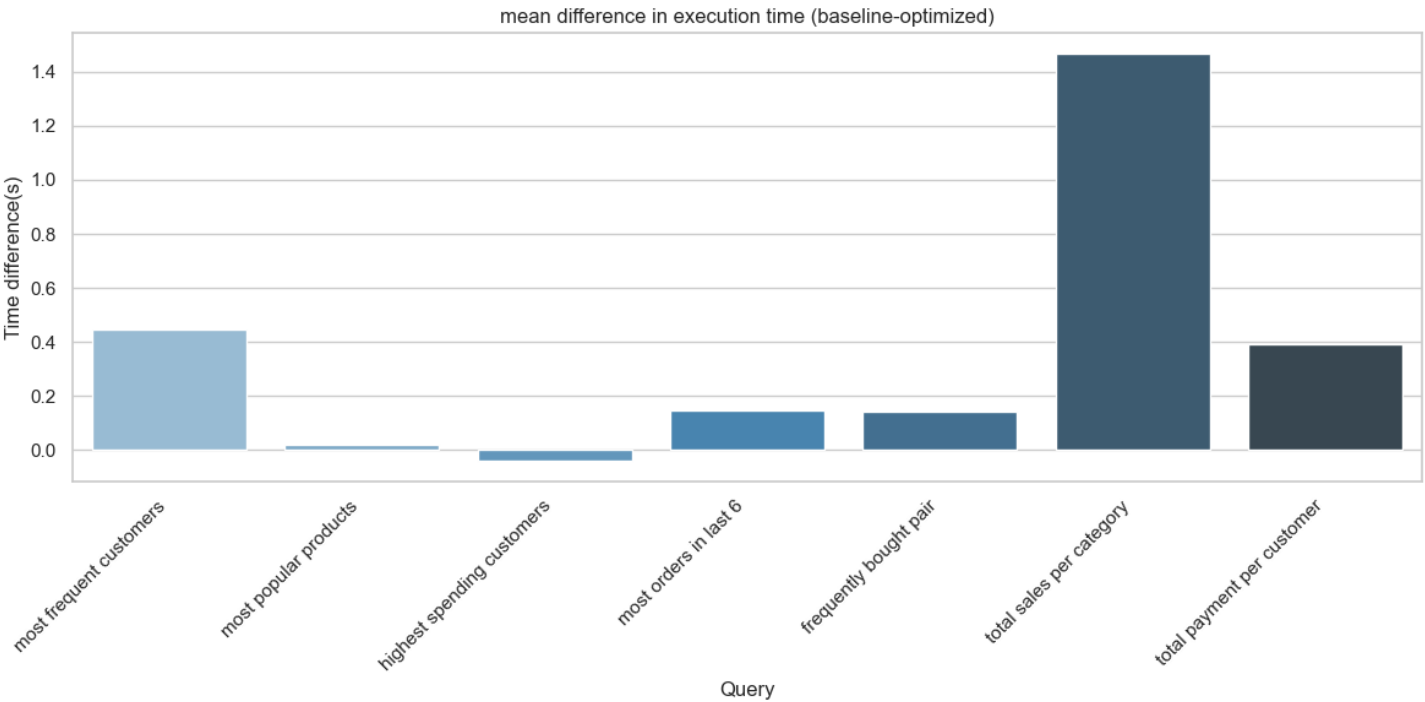


Fig 13: Mean difference in execution time

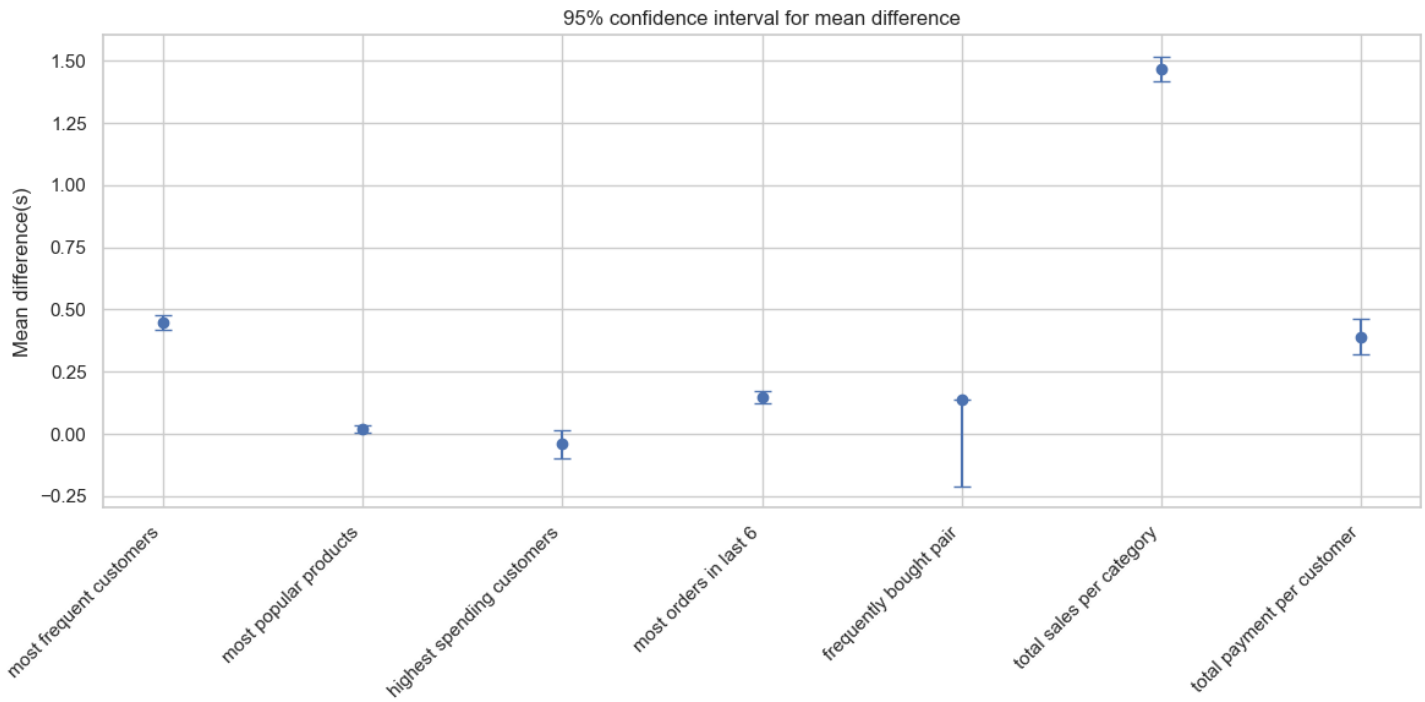


Fig 14: 95% confidence interval for mean difference

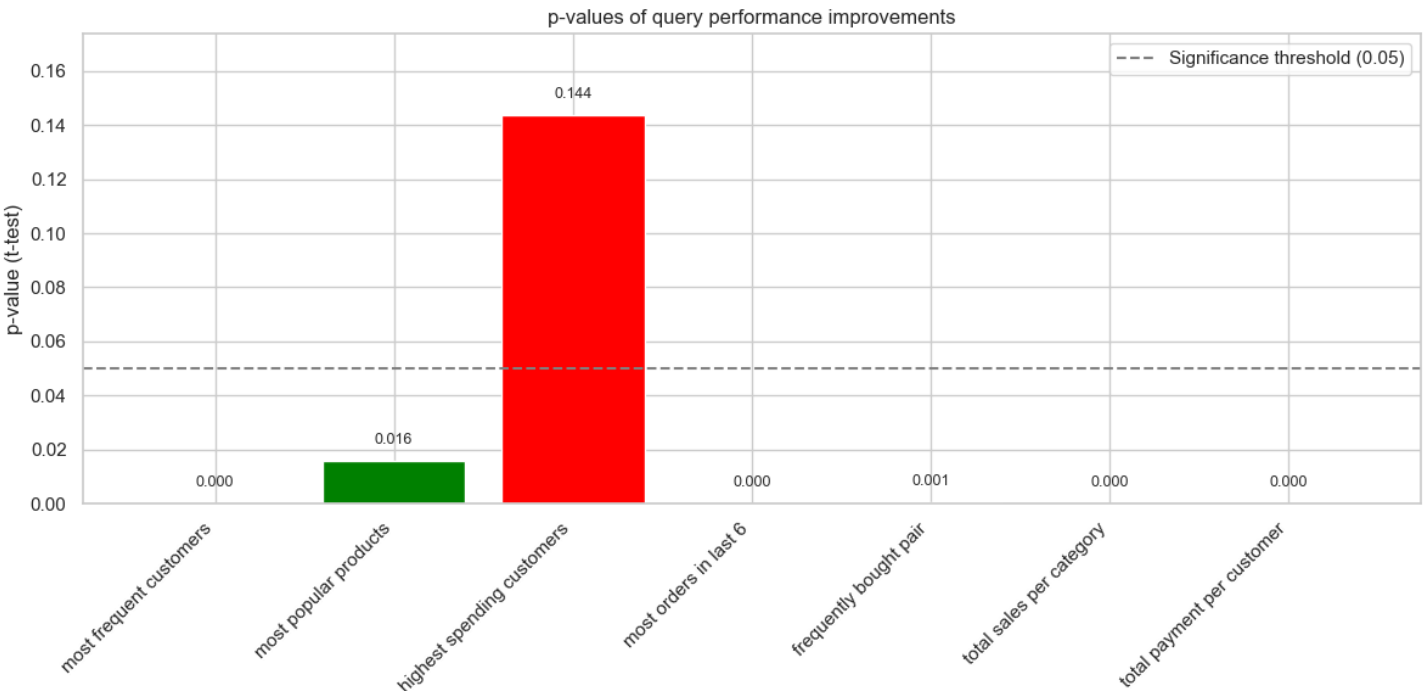


Fig 15: P-values

3.4 Query performance before vs after indexing

I compared the query performance (execution time and statistical significance) before and after adding single, composite, and partial indexes.

3.4.1 Execution time comparison

Query name	Optimized avg (before indexing)	Optimized avg (after indexing)	Improvement (after vs before)
most frequent customers	0.0006	0.0003	50% faster than already optimized
Most popular products	0.9294	0.1571	83% faster
Highest spending customers	1.8676	1.2707	32% faster
Customers with most orders in 6 months	0.4447	0.1964	55% faster
Most frequently bought product pair	3.3568	2.8762	14.3% faster
Total sales per category	1.246	0.968	22.3% faster
Total payment per customer	1.3876	0.7222	48% faster

Table 1: Execution time comparison results

3.4.2 Statistical significance comparison

Query name	Mean diff before indexing (B-O)	p-value before indexing	Mean diff after indexing (B-O)	p-value after indexing	Performance analysis
most frequent customers	0.6721	0.000	0.4478	0.000	Still significant
Most popular products	0.12	0.01768	0.0182	0.01583	Both significant, but indexing improved range
Highest spending customers	0.1765	0.000682	-0.0407	0.1436	No longer significant after indexing
Customers with most orders in 6 months	0.0017	0.936	0.1466	0.000	Became significant after indexing
Most frequently bought product pair	0.3758	0.01539	-0.1393	0.0006	Became slower after indexing
Total sales per category	1.833	0.000	1.4676	0.000	Both significantly improved
Total payment per customer	0.1988	0.02723	0.3899	0.000	More significant after indexing

Table 2: Statistical significance comparison results

Indexing made more queries statistically significant by reducing variability and speed gains. A few queries, like highest spending customers showed reduced statistical confidence, may be for their dependency on aggregation over joined tables that didn’t directly benefit from the indexes. product pair queries became significantly slower. It's a sign that index overhead for complex self-joins might surpass benefits in this case.

4. Transaction handling and concurrency

Single-session transaction handling:

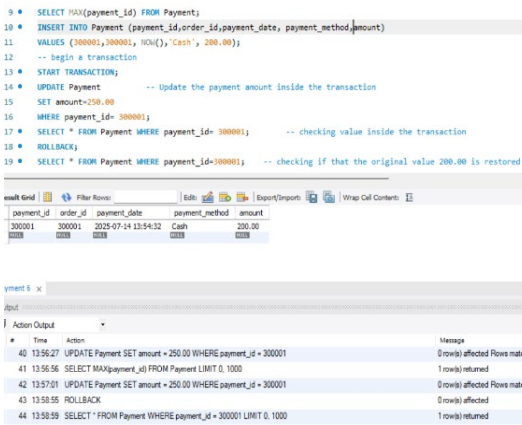


Fig 16: Using ROLLBACK

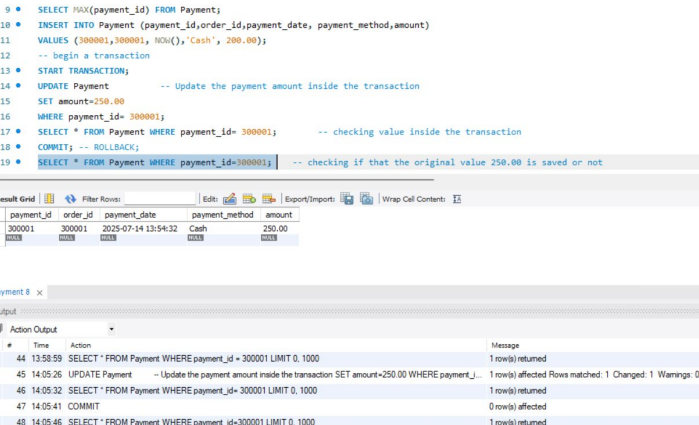


Fig 17: Using COMMIT

In Fig 1, using ROLLBACK

Steps:

- Inserted a new payment row with amount=200.00
- Started a transaction
- Updated amount =300.00
- Checked the updated value (inside the transaction)
- Executed ROLLBACK
- Checked again -> the value reverted to 200.00

In Fig 2, using COMMIT

Steps:

- Again, started a transaction
- Updated amount = 250.00
- Checked value (shows 250.00 inside the transaction)
- Used COMMIT
- Verified final value -> still 250.00

4.1 Illustrating three anomalies

- 1. Dirty read: Reading a value that another transaction hasn't committed yet.
- 2. Non-Repeatable read: Reading the same row twice and getting different results.
- 3. Phantom read: Reading a group of rows twice and getting new rows the second time.

	Phantom read	Nonrepeatable read	Dirty read
SERIALIZABLE	Not allowed	Not allowed	Not allowed
REPEATABLE READ	Allowed	Not allowed	Not allowed
READ COMMITTED	Allowed	Allowed	Not allowed
READ UNCOMMITTED	Allowed	Allowed	Allowed

Fig 18: Transaction isolation levels and the anomalies they allow

4.1.1 Dirty read (READ UNCOMMITTED)

In my dirty read simulation, Session A first read the committed value 999.00. Then, Session B performed an update to amount = 555.00 inside a transaction but did not commit. When Session A queried the same row again (still using READ UNCOMMITTED), it observed the uncommitted value 555.00. This confirmed the occurrence of a dirty read, where uncommitted changes from another transaction are visible. This behavior is prevented in higher isolation levels like READ COMMITTED.

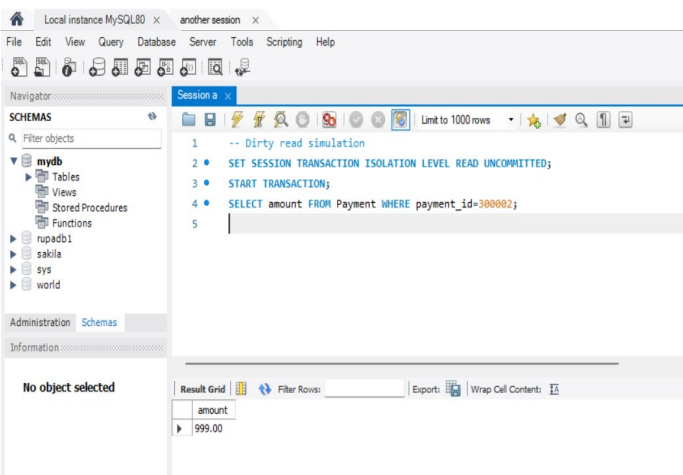


Fig 19: Session A

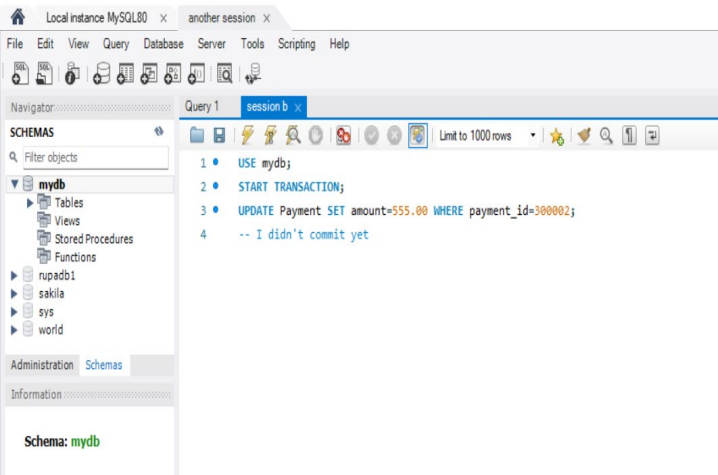


Fig 20: Session B

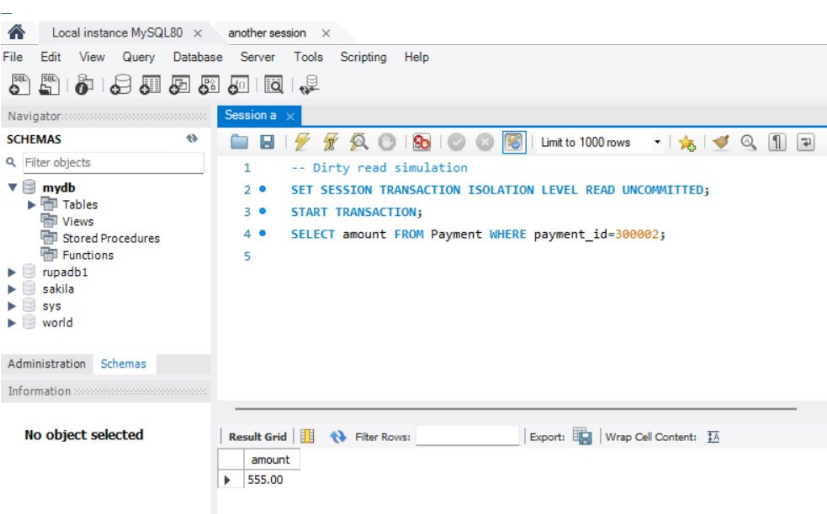


Fig 21: Session A

4.1.2 Non-repeatable read (READ COMMITTED)

I simulated a non-repeatable read by first reading a row in Session A under READ COMMITTED, then updating and committing that same row in Session B. When Session A queried the row again, the value had changed from 999.00 to 555.00. This showed a non-repeatable read anomaly- the same read query returned different results during the same transaction. It can be prevented by using the REPEATABLE READ isolation level.

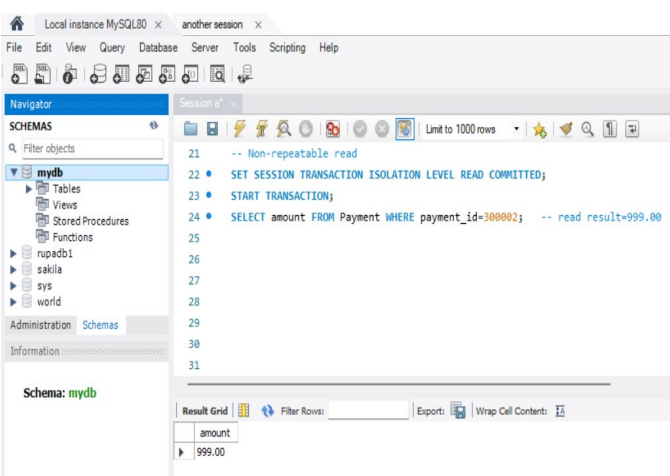


Fig 22: Session A

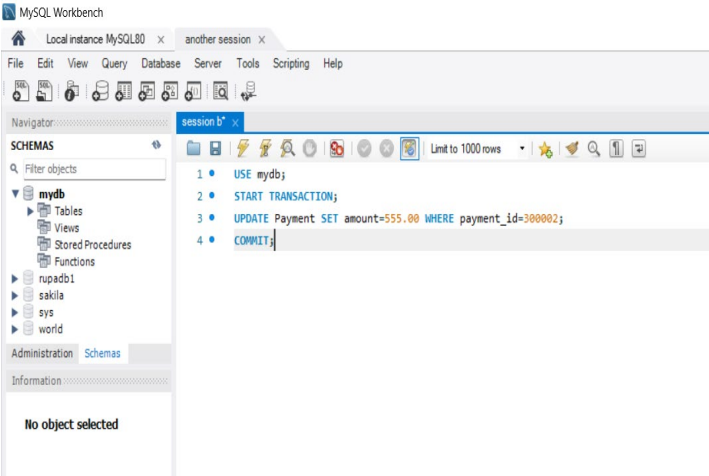


Fig 23: Session B

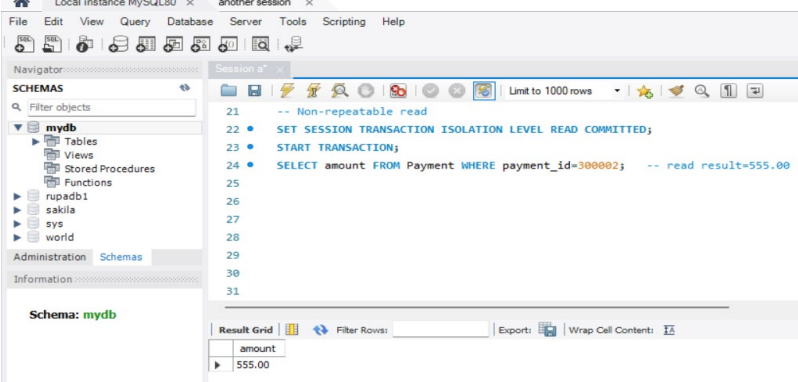


Fig 24: Session A

4.1.3 Phantom read (READ COMMITTED)

I tested a phantom read by first counting how many payments were greater than 200 using the READ COMMITTED setting. While that session (Session A) was still running, another session (Session B) added a new payment that also had an amount over 200. When Session A ran the same count query again, the number had gone up. This happened even though Session A didn't restart its transaction. That change in results shows a phantom read, where new rows appear during a transaction because of another user's actions. To stop this from happening, the SERIALIZABLE level can be used, which would block the new insert until Session A is finished.

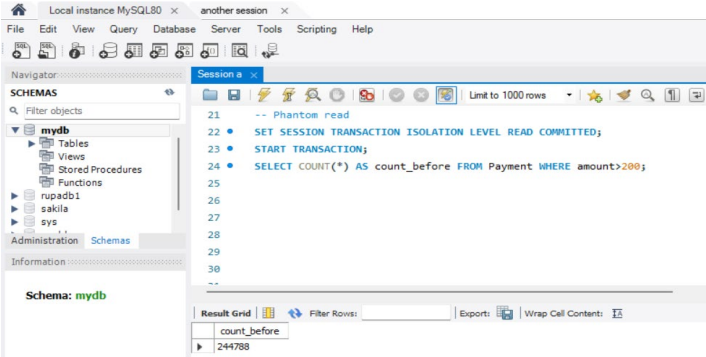


Fig 25: Session A

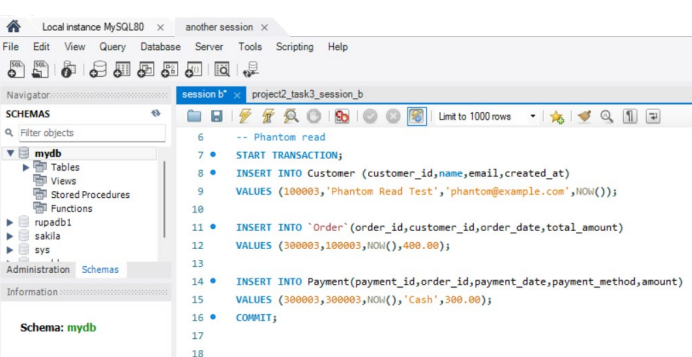


Fig 26: Session B

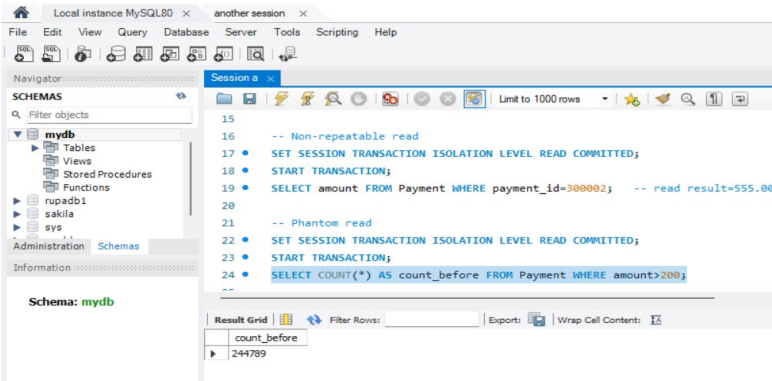


Fig 27: Session A

4.2 Preventing concurrency anomalies

4.2.1 Preventing dirty read using READ COMMITTED

I tested how READ COMMITTED prevents dirty reads by simulating a concurrent update in Session B. The update to amount = 999.00 was issued without a COMMIT. In Session A, using the READ COMMITTED isolation level, I queried the same row but only observed the previously committed value 555.00. This confirmed that uncommitted changes from concurrent transactions remain hidden, effectively preventing dirty reads.

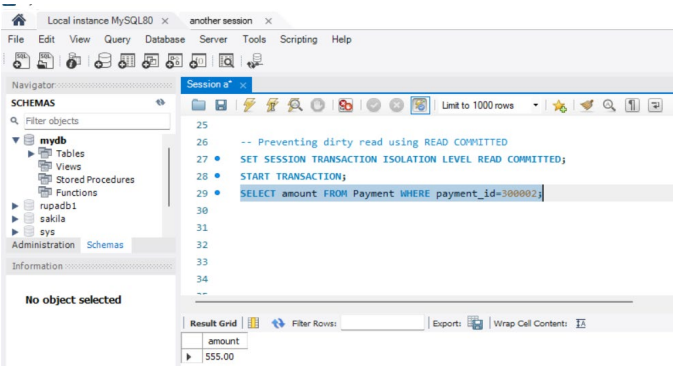


Fig 28: Session A

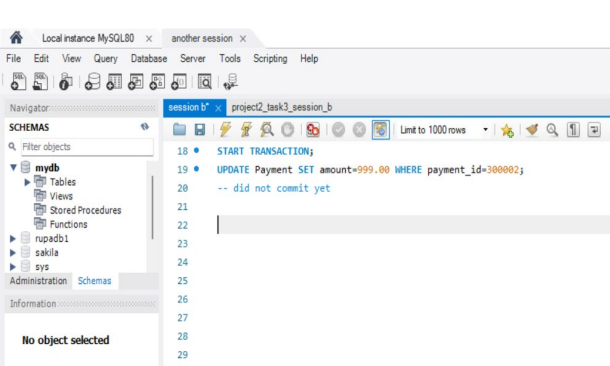


Fig 29: Session B

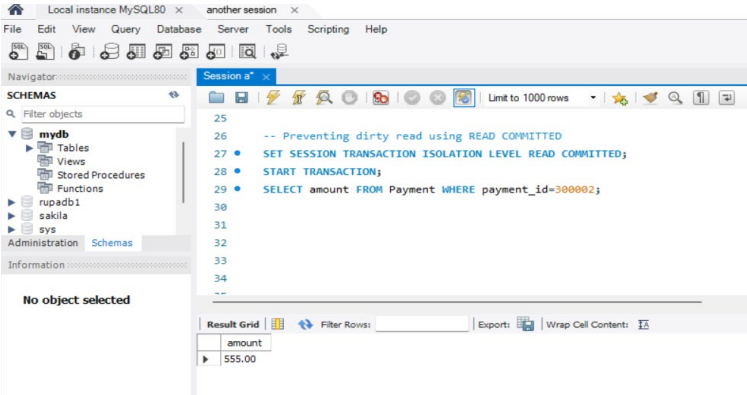


Fig 30: Session A

4.2.2 Preventing non-repeatable read using REPEATABLE READ

I prevented a non-repeatable read using the REPEATABLE READ isolation level. Session A read a row's value as 555.00, while Session B concurrently updated it to 666.00 and committed. When Session A read the row again within the same transaction, it still saw 555.00. This behavior confirms that REPEATABLE READ maintains a consistent snapshot of data across multiple reads in the same transaction, preventing non-repeatable read anomalies.

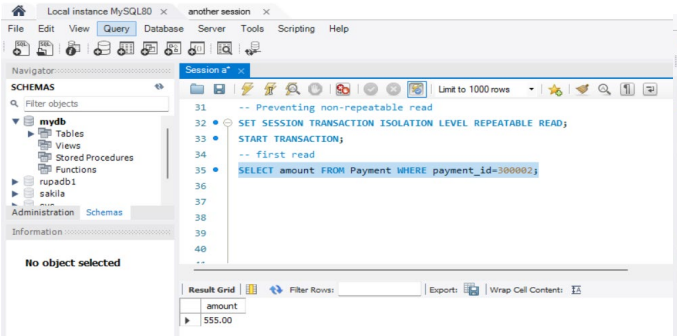


Fig 31: Session A

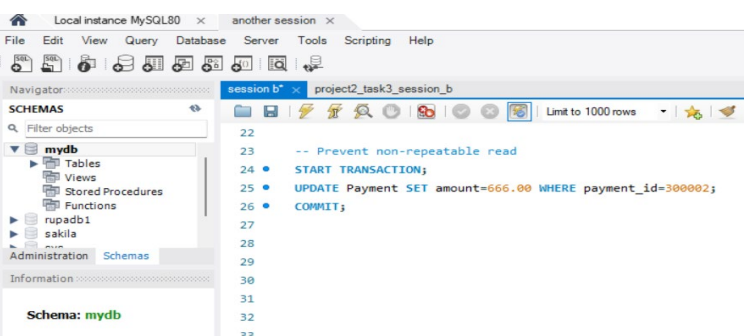


Fig 32: Session B

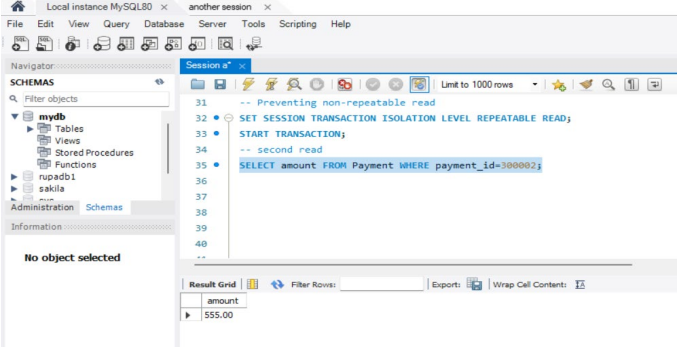


Fig 33: Session A

4.2.3 Preventing phantom read using SERIALIZABLE

I used the SERIALIZABLE isolation level to stop phantom reads. In Session A, I ran a query to count how many payments were greater than 200. While that transaction was still open, Session B tried to insert a new payment with an amount 300.00, which also falls into that range. Because Session A was running in SERIALIZABLE mode, MySQL placed a lock on that range. As a result, Session B's insert was blocked until Session A finished, proving that phantom reads were successfully prevented.

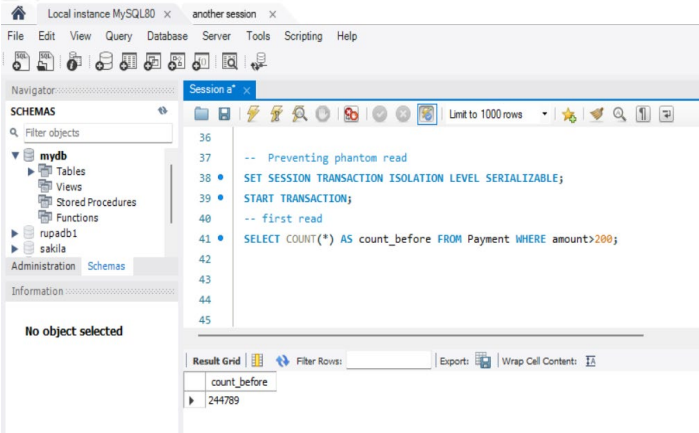


Fig 34: Session A

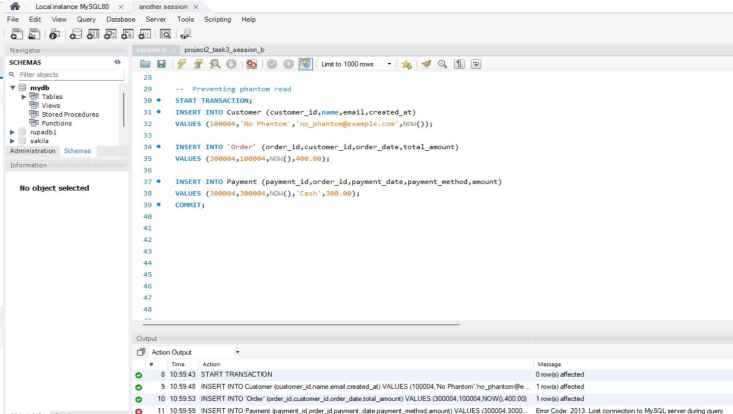


Fig 35: Session B